NAME:ROHAN RESHI

PRN:202401040034

BATCH:A1-6

IA ASSIGNMENT 2

GITHUB LINK : https://github.com/astarohan/AIML-TH-PEC

Ecommerce Data Analysis Using Web Scraping and Feature Engineering

Aim

To collect product information using web scraping and apply feature engineering techniques to clean, transform, analyze, and visualize an ecommerce dataset.

Problem Statement

Ecommerce data contains useful information about product prices, ratings, reviews, and availability. Collecting information manually is time-consuming, and raw datasets may contain missing values, duplicate records, inconsistent formats, categorical variables, and outliers. Web scraping can automate data collection, while feature engineering can improve data quality and prepare the dataset for analysis.

Objectives

1. To extract product information from a website using Python.
2. To organize scraped data into a structured DataFrame.
3. To inspect missing values and duplicate records in the original dataset.
4. To clean and convert product-related numerical features.
5. To handle missing values using median imputation.
6. To encode categorical and binary features.
7. To create discount amount and discount percentage features.
8. To calculate standard deviation and understand data variability.
9. To detect and cap outliers using the IQR method.
10. To scale numerical features using StandardScaler.
11. To visualize data using histograms, box plots, bar charts, scatter plots, line charts, pair plots, and correlation heatmaps.
12. To save the processed dataset for further analysis.

```
#  Import libraries and connect Google Drive

import os
import re
import time
import numpy as np
import pandas as pd
import requests
import matplotlib.pyplot as plt
import seaborn as sns

from bs4 import BeautifulSoup
from google.colab import drive
from sklearn.preprocessing import StandardScaler
```

```
from IPython.display import display

drive.mount('/content/drive')

folder_path = '/content/drive/MyDrive/Dataset'
os.makedirs(folder_path, exist_ok=True)

print("Libraries imported and Google Drive connected.")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/dri
Libraries imported and Google Drive connected.
```

Web Scraping — Theory

Web scraping is the process of automatically collecting information from websites using programming. Python libraries such as Requests retrieve website content, while BeautifulSoup parses HTML and helps extract required information. The extracted information can be organized into a Pandas DataFrame for further analysis.

```
#  Send a request to the practice website

base_url = 'https://books.toscrape.com/'
first_url = 'https://books.toscrape.com/catalogue/page-1.html'

headers = {'User-Agent': 'Mozilla/5.0'}

response = requests.get(first_url, headers=headers, timeout=20)

print("HTTP status:", response.status_code)

if response.status_code == 200:
    soup = BeautifulSoup(response.text, 'html.parser')
    print("Website accessed successfully.")
    print("Page title:", soup.title.get_text(strip=True))
else:
    raise RuntimeError("Website request failed. Check the URL or retry.")
```

```
HTTP status: 200
Website accessed successfully.
Page title: All products | Books to Scrape - Sandbox
```

Insight

This cell checks whether the website can be accessed and retrieves its HTML content. An HTTP status code of 200 generally indicates a successful request. The retrieved HTML will be used to extract product information in the next cell.

```
# Cell 6: Extract product information from one page

def extract_products(page_soup):
    products = []

    for item in page_soup.select('article.product_pod'):
        title_tag = item.select_one('h3 a')
        price_tag = item.select_one('.price_color')
        availability_tag = item.select_one('.availability')
        rating_tag = item.select_one('p.star-rating')

        title = title_tag.get('title', '').strip() if title_tag else None
        price = price_tag.get_text(strip=True) if price_tag else None

        availability = (
            availability_tag.get_text(" ", strip=True)
```

```
            if availability_tag else None
        )

        rating = (
            rating_tag.get('class', [])[-1]
            if rating_tag and len(rating_tag.get('class', [])) > 1
            else None
        )

        relative_url = title_tag.get('href') if title_tag else None
        product_url = (
            requests.compat.urljoin(first_url, relative_url)
            if relative_url else None
        )

        products.append({
            'title': title,
            'price': price,
            'rating': rating,
            'availability': availability,
            'product_url': product_url
        })

    return products


first_page_products = extract_products(soup)

print("Products extracted:", len(first_page_products))
display(pd.DataFrame(first_page_products).head(10))
```

```
Products extracted: 20
```

	title	price	rating	availability	product_url
0	A Light in the Attic	Â£51.77	Three	In stock	https://books.toscrape.com/catalogue/a-light-i...
1	Tipping the Velvet	Â£53.74	One	In stock	https://books.toscrape.com/catalogue/tipping-t...
2	Soumission	Â£50.10	One	In stock	https://books.toscrape.com/catalogue/soumissio...
3	Sharp Objects	Â£47.82	Four	In stock	https://books.toscrape.com/catalogue/sharp-obj...
4	Sapiens: A Brief History of Humankind	Â£54.23	Five	In stock	https://books.toscrape.com/catalogue/sapiens-a...
5	The Requiem Red	Â£22.65	One	In stock	https://books.toscrape.com/catalogue/the-requi...
6	The Dirty Little Secrets of Getting Your Dream...	Â£33.34	Four	In stock	https://books.toscrape.com/catalogue/the-dirty...
7	The Coming Woman: A Novel Based on the Life of...	Â£17.93	Three	In stock	https://books.toscrape.com/catalogue/the-comin...

Product Extraction — Theory and Insight

The HTML elements for each product are located using BeautifulSoup selectors. The code extracts product titles, prices, written star ratings, availability, and product URLs.

The displayed table shows the extracted records and helps verify that the extraction process is working before collecting information from multiple pages.

```
#  Save scraped data separately

scraped_path = os.path.join(
    folder_path, 'web_scraped_ecommerce_data.csv'
)
```

```
if not scraped_df.empty:
    scraped_df.to_csv(scraped_path, index=False)

    print("Scraped data saved to:", scraped_path)
    print("Dataset shape:", scraped_df.shape)
    display(scraped_df.head(10))
else:
    print("No products were scraped. Check the website request.")
```

```
Scraped data saved to: /content/drive/MyDrive/Dataset/web_scraped_ecommerce_data.csv
Dataset shape: (1000, 5)
```

	title	price	rating	availability	product_url
0	A Light in the Attic	Â£51.77	Three	In stock	https://books.toscrape.com/catalogue/a-light-i...
1	Tipping the Velvet	Â£53.74	One	In stock	https://books.toscrape.com/catalogue/tipping-t...
2	Soumission	Â£50.10	One	In stock	https://books.toscrape.com/catalogue/soumissio...
3	Sharp Objects	Â£47.82	Four	In stock	https://books.toscrape.com/catalogue/sharp-obj...
4	Sapiens: A Brief History of Humankind	Â£54.23	Five	In stock	https://books.toscrape.com/catalogue/sapiens-a...
5	The Requiem Red	Â£22.65	One	In stock	https://books.toscrape.com/catalogue/the-requi...
6	The Dirty Little Secrets of Getting Your Dream...	Â£33.34	Four	In stock	https://books.toscrape.com/catalogue/the-dirty...
7	The Coming Woman: A Novel Based on the Life of...	Â£17.93	Three	In stock	https://books.toscrape.com/catalogue/the-comin...

Insight — Saving Scraped Data

The scraped product records are saved as a CSV file in Google Drive. This allows the data to be reused without scraping the website again. Because this practice website contains books, this CSV remains separate from the original ecommerce product dataset used in the feature engineering section.

Feature Engineering — Theory

Feature engineering is the process of transforming existing data and creating useful features for analysis and machine learning. It includes data cleaning, handling missing values, encoding categorical variables, creating new features, treating outliers, and scaling numerical data.

In this section, the original ecommerce dataset is used. The dataset is inspected, processed, visualized, and saved separately from the scraped practice data.

```
#  Load the original ecommerce dataset

file_path = os.path.join(
    folder_path, 'ecommerce_product_dataset_50000.csv'
)

if not os.path.exists(file_path):
    print("CSV files available in your Dataset folder:")
    print([f for f in os.listdir(folder_path) if f.endswith('.csv')])
    raise FileNotFoundError(
        "Check the filename above and update file_path."
    )

df = pd.read_csv(file_path)

# Remove accidental spaces from column names
df.columns = df.columns.astype(str).str.strip()
```

```
print("Original dataset loaded successfully.")
print("Dataset shape:", df.shape)
display(df.head())
```

```
Original dataset loaded successfully.
Dataset shape: (50010, 18)
```

	title	category	brand	rating	number_of_reviews	bought_in_last_month	current/discounted_pr
0	Orion Yoga Mat Model 1	Toys	Orion	4.0 out of 5 stars	182	NaN	8
1	Nova Gaming Mouse Model 2	Home & Kitchen	Nova	4.2 out of 5 stars	22	517+ bought in past month	1
2	Pulse Water Bottle Model 3	Electronics	Pulse	4.5 out of 5 stars	1,294	1.3K+ bought in past month	7
3	Orion Fitness Band Model 4	Books	Orion	4.5 out of 5 stars	146	660+ bought in past month	2
4	Zenith Wireless Headphones Model 5	Clothing	Zenith	3.4 out of 5 stars	1,024	187+ bought in past month	1

```
#  Inspect structure, missing values, and duplicates

print("Dataset information:")
df.info()

print("\nMissing values per column:")
display(df.isnull().sum().to_frame('Missing Values'))

print("\nTotal duplicate rows:", df.duplicated().sum())

# Remove exact duplicate rows once
df = df.drop_duplicates().copy()

print("\nShape after removing duplicate rows:", df.shape)
```

```
Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50010 entries, 0 to 50009
Data columns (total 18 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   title                     50010 non-null  object
 1   category                  50010 non-null  object
 2   brand                     50010 non-null  object
 3   rating                    48777 non-null  object
 4   number_of_reviews         48826 non-null  object
 5   bought_in_last_month      46576 non-null  object
 6   current/discounted_price  44010 non-null  float64
 7   price_on_variant          50010 non-null  object
 8   listed_price              50010 non-null  object
 9   is_best_seller            50010 non-null  object
 10  is_sponsored              50010 non-null  object
 11  is_couponed               50010 non-null  object
 12  buy_box_availability      35831 non-null  object
 13  delivery_details          35284 non-null  object
 14  sustainability_badges     790 non-null    object
 15  image_url                 50010 non-null  object
 16  product_url               48003 non-null  object
 17  collected_at              50010 non-null  object
dtypes: float64(1), object(17)
memory usage: 6.9+ MB

Missing values per column:
```

	Missing Values
title	0
category	0
brand	0
rating	1233
number_of_reviews	1184
bought_in_last_month	3434
current/discounted_price	6000
price_on_variant	0
listed_price	0
is_best_seller	0
is_sponsored	0
is_couponed	0
buy_box_availability	14179
delivery_details	14726
sustainability_badges	49220
image_url	0
product_url	2007
collected_at	0

```
Total duplicate rows: 10

Shape after removing duplicate rows: (50000, 18)
```

Insight — Dataset Inspection

This cell displays the dataset structure, data types, missing-value counts, and duplicate count. Exact duplicate rows are removed to avoid counting identical records more than once. The output helps identify the data-

cleaning requirements before feature engineering begins.

```
# Cell 16: Convert source columns into numerical features

def find_column(candidates):
    # Exact match first
    for candidate in candidates:
        if candidate in df.columns:
            return candidate

    # Case-insensitive match
    normalized = {
        str(col).strip().lower(): col for col in df.columns
    }

    for candidate in candidates:
        key = candidate.strip().lower()
        if key in normalized:
            return normalized[key]

    return None


def parse_number(value):
    if pd.isna(value):
        return np.nan

    text = str(value).replace(',', '').strip()
    match = re.search(r'-?\d+(?:\.\d+)?', text)

    return float(match.group()) if match else np.nan


def parse_count(value):
    if pd.isna(value):
        return np.nan

    text = str(value).strip().replace(',', '')
    match = re.search(r'(\d+(?:\.\d+)?)\s*([KkMm]?)', text)

    if not match:
        return np.nan

    number = float(match.group(1))
    suffix = match.group(2).lower()

    if suffix == 'k':
        number *= 1000
    elif suffix == 'm':
        number *= 1000000

    return number


# Ratings such as "4.6 out of 5 stars" or "4.6"
rating_col = find_column(['rating', 'ratings'])

if rating_col:
    df['rating_num'] = df[rating_col].apply(parse_number)

# Review counts
reviews_col = find_column([
    'number_of_reviews', 'reviews', 'review_count'
])

if reviews_col:
    df['reviews_num'] = df[reviews_col].apply(parse_count)
```

```
# Current/discounted price
current_price_col = find_column([
    'current/discounted_price',
    'current_price',
    'discounted_price',
    'current price',
    'price'
])

if current_price_col:
    df['current_price_num'] = df[current_price_col].apply(parse_number)

# Listed/original price
listed_price_col = find_column([
    'listed_price', 'original_price', 'mrp', 'list_price'
])

if listed_price_col:
    df['listed_price_num'] = df[listed_price_col].apply(parse_number)

# Monthly purchases
bought_col = find_column([
    'bought_in_last_month',
    'bought_last_month',
    'purchased_last_month'
])

if bought_col:
    df['bought_last_month_num'] = df[bought_col].apply(parse_count)

created_cols = [
    col for col in [
        'rating_num',
        'reviews_num',
        'current_price_num',
        'listed_price_num',
        'bought_last_month_num'
    ] if col in df.columns
]

print("Numerical features created:", created_cols)
display(df[created_cols].head())
```

```
Numerical features created: ['rating_num', 'reviews_num', 'current_price_num', 'listed_price_num', 'bou
```

	rating_num	reviews_num	current_price_num	listed_price_num	bought_last_month_num
0	4.0	182.0	84.54	105.67	NaN
1	4.2	22.0	11.32	18.86	517.0
2	4.5	1294.0	71.00	NaN	1300.0
3	4.5	146.0	23.90	29.87	660.0
4	3.4	1024.0	11.74	NaN	187.0

Insight — Numerical Conversion

Product ratings, prices, review counts, and monthly purchase counts may be stored as text. This cell converts available source columns into numerical features so that statistical calculations, outlier detection, and visualizations can be performed. If a feature is not created, check the source column names in the dataset.

```
#  Median imputation for available numerical features

cols_to_impute = [
```

```
    col for col in [
        'rating_num',
        'reviews_num',
        'current_price_num',
        'listed_price_num',
        'bought_last_month_num'
    ]
    if col in df.columns
]

print("Missing values before imputation:")
display(df[cols_to_impute].isnull().sum())

for col in cols_to_impute:
    median_val = df[col].median()

    if pd.notna(median_val):
        df[col] = df[col].fillna(median_val)
        print(f"{col}: median used = {median_val:.2f}")
    else:
        print(f"{col}: all values missing; median imputation skipped.")

print("\nMissing values after imputation:")
display(df[cols_to_impute].isnull().sum())
```

Insight — Median Imputation

Median imputation replaces missing numerical values with the median of the corresponding column. The median is less affected by extreme values than the mean. Comparing missing-value counts before and after imputation shows whether the missing numerical values have been handled successfully.

```
# Cell 20: Calculate standard deviation

print("Standard deviation of numerical features:")
display(df[cols_to_impute].std())
```

```
Standard deviation of numerical features:
```

	0
rating_num	0.602989
reviews_num	1296.138806
current_price_num	44.017240
listed_price_num	52.351683
bought_last_month_num	635.322349

dtype: float64

Standard Deviation

Standard deviation measures how much values vary around their mean. A low standard deviation indicates that values are relatively close to the mean, while a high standard deviation indicates greater variation.

Insight: Standard deviation helps compare the variability of ratings, prices, reviews, and purchase counts. Since these features use different units and ranges, their standard deviations should be interpreted in the context of each feature.

```
# Cell 22: Encode available categorical and binary features

# Encode common yes/no and true/false fields
```

```
binary_candidates = {
    'is_best_seller': ['is_best_seller', 'best_seller'],
    'is_sponsored': ['is_sponsored', 'sponsored'],
    'is_couponed': ['is_couponed', 'couponed']
}

def to_binary(value):
    if pd.isna(value):
        return np.nan

    text = str(value).strip().lower()

    if text in ['yes', 'true', '1', 'y']:
        return 1
    if text in ['no', 'false', '0', 'n']:
        return 0

    return np.nan


for new_col, candidates in binary_candidates.items():
    source_col = find_column(candidates)

    if source_col:
        df[new_col + '_bin'] = df[source_col].apply(to_binary)
        print(f"Encoded {source_col} as {new_col}_bin")

# One-hot encode a category column if present
category_col = find_column(['category', 'product_category'])

if category_col:
    df['category_encoded'] = (
        df[category_col].astype('category').cat.codes
    )
    print("Encoded category column:", category_col)

display(df.head())
```

```
Encoded is_best_seller as is_best_seller_bin
Encoded is_sponsored as is_sponsored_bin
Encoded is_couponed as is_couponed_bin
Encoded category column: category
```

	title	category	brand	rating	number_of_reviews	bought_in_last_month	current/discounted_pr
0	Orion Yoga Mat Model 1	Toys	Orion	4.0 out of 5 stars	182	NaN	8
1	Nova Gaming Mouse Model 2	Home & Kitchen	Nova	4.2 out of 5 stars	22	517+ bought in past month	1
2	Pulse Water Bottle Model 3	Electronics	Pulse	4.5 out of 5 stars	1,294	1.3K+ bought in past month	7
3	Orion Fitness Band Model 4	Books	Orion	4.5 out of 5 stars	146	660+ bought in past month	2
4	Zenith Wireless Headphones Model 5	Clothing	Zenith	3.4 out of 5 stars	1,024	187+ bought in past month	1

```
# Cell 24: Create discount amount and discount percentage
```

```
if (
    'current_price_num' in df.columns
    and 'listed_price_num' in df.columns
):
    df['discount_amount'] = (
        df['listed_price_num'] - df['current_price_num']
    ).clip(lower=0)

    df['discount_percentage'] = np.where(
        df['listed_price_num'] > 0,
        (df['discount_amount'] / df['listed_price_num']) * 100,
        np.nan
    )

    print("Discount features created successfully.")
    display(df[
        [
            'current_price_num',
            'listed_price_num',
            'discount_amount',
            'discount_percentage'
        ]
    ].head())
else:
    print(
        "Discount features could not be created. "
        "Check whether both price columns exist."
    )
```

```
Discount features created successfully.
```

	current_price_num	listed_price_num	discount_amount	discount_percentage
0	84.54	105.67	21.13	19.996215
1	11.32	18.86	7.54	39.978791
2	71.00	NaN	NaN	NaN
3	23.90	29.87	5.97	19.986609
4	11.74	NaN	NaN	NaN

```
# Cell 26: Detect and cap outliers using IQR

numeric_cols_for_outliers = [
    col for col in [
        'reviews_num',
        'current_price_num',
        'bought_last_month_num'
    ]
    if col in df.columns
    and pd.api.types.is_numeric_dtype(df[col])
    and df[col].notna().any()
]

if not numeric_cols_for_outliers:
    print("No suitable numerical features found for outlier treatment.")
else:
    # Box plots before capping
    fig, axes = plt.subplots(
        1, len(numeric_cols_for_outliers),
        figsize=(5 * len(numeric_cols_for_outliers), 5)
    )

    if len(numeric_cols_for_outliers) == 1:
        axes = [axes]

    for ax, col in zip(axes, numeric_cols_for_outliers):
```

```
    sns.boxplot(y=df[col], ax=ax)
    ax.set_title(f'Before Capping: {col}')
    ax.set_ylabel('Value')

plt.tight_layout()
plt.show()

# Cap outliers using IQR
for col in numeric_cols_for_outliers:
    q1 = df[col].quantile(0.25)
    q3 = df[col].quantile(0.75)
    iqr = q3 - q1

    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr

    df[col] = df[col].clip(
        lower=lower_bound,
        upper=upper_bound
    )

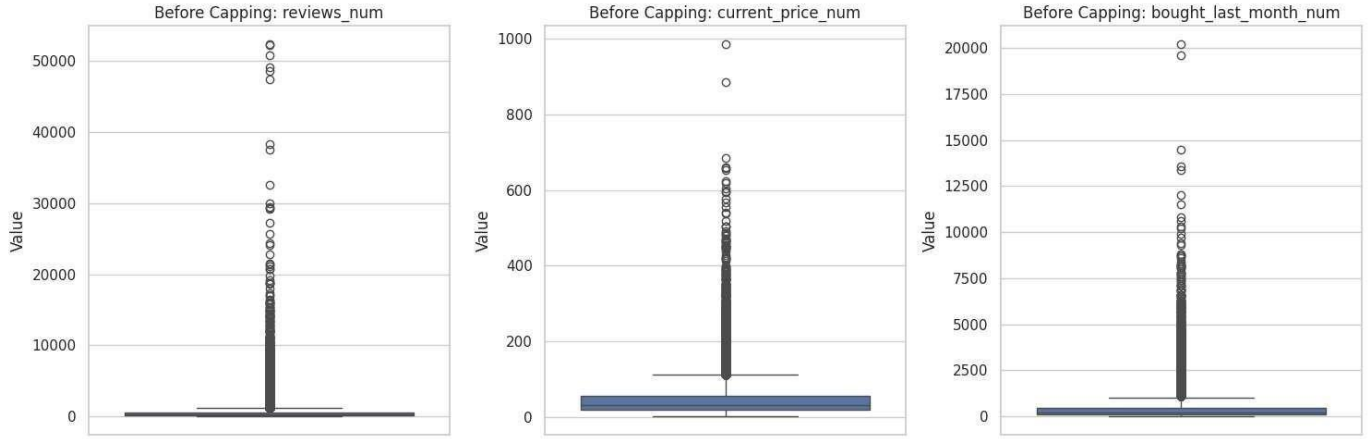
    print(
        f"{col}: lower limit = {lower_bound:.2f}, "
        f"upper limit = {upper_bound:.2f}"
    )

# Box plots after capping
fig, axes = plt.subplots(
    1, len(numeric_cols_for_outliers),
    figsize=(5 * len(numeric_cols_for_outliers), 5)
)

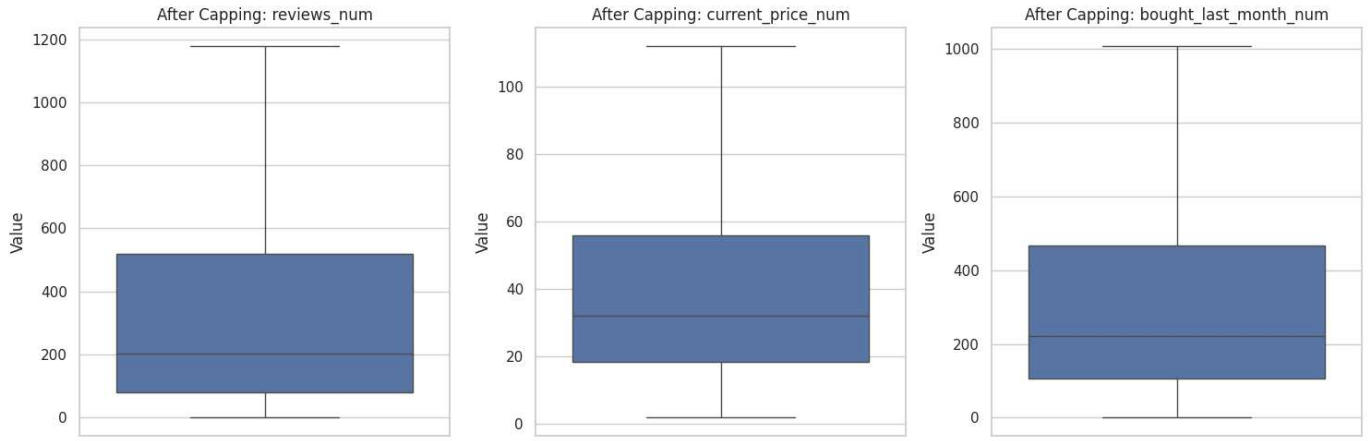
if len(numeric_cols_for_outliers) == 1:
    axes = [axes]

for ax, col in zip(axes, numeric_cols_for_outliers):
    sns.boxplot(y=df[col], ax=ax)
    ax.set_title(f'After Capping: {col}')
    ax.set_ylabel('Value')

plt.tight_layout()
plt.show()
```




```
reviews_num: lower limit = -582.00, upper limit = 1178.00
current_price_num: lower limit = -37.48, upper limit = 111.78
bought_last_month_num: lower limit = -436.50, upper limit = 1007.50
```



Outlier Detection Using IQR

The Interquartile Range (IQR) is calculated as Q3 – Q1, where Q1 is the first quartile and Q3 is the third quartile. The usual lower and upper limits are Q1 – 1.5 × IQR and Q3 + 1.5 × IQR.

Insight: The before-and-after box plots show how capping limits extreme values. Capping retains the records but changes values outside the calculated limits. Some points may still be shown at the limits because multiple observations can share the same capped value.

```
# Cell 28: Apply StandardScaler

scale_candidates = [
    'rating_num',
    'reviews_num',
```

```
    'current_price_num',
    'listed_price_num',
    'bought_last_month_num',
    'discount_amount',
    'discount_percentage'
]

scale_cols = [
    col for col in scale_candidates
    if col in df.columns
    and pd.api.types.is_numeric_dtype(df[col])
    and df[col].notna().any()
]

df_scaled_features = pd.DataFrame(index=df.index)

if scale_cols:
    scaler = StandardScaler()

    # Fill remaining missing values temporarily for scaling
    scale_input = df[scale_cols].copy()
    scale_input = scale_input.fillna(scale_input.median())

    scaled_values = scaler.fit_transform(scale_input)

    df_scaled_features = pd.DataFrame(
        scaled_values,
        columns=[col + '_scaled' for col in scale_cols],
        index=df.index
    )

    print("Scaled features:")
    display(df_scaled_features.head())
else:
    print("No numerical features available for scaling.")
```

```
Scaled features:
```

	rating_num_scaled	reviews_num_scaled	current_price_num_scaled	listed_price_num_scaled	bought_las
0	-0.125917	-0.472093	1.577234	1.076553	
1	0.209924	-0.903270	-1.024317	-0.713367	
2	0.713684	2.211983	1.096149	-0.262433	
3	0.713684	-0.569108	-0.577342	-0.486353	
4	-1.133437	1.796975	-1.009394	-0.262433	

Feature Scaling Using StandardScaler

StandardScaler transforms numerical features so that each feature has a mean close to 0 and a standard deviation close to 1 in the data used to fit the scaler. Scaling is useful when features have different units or magnitudes.

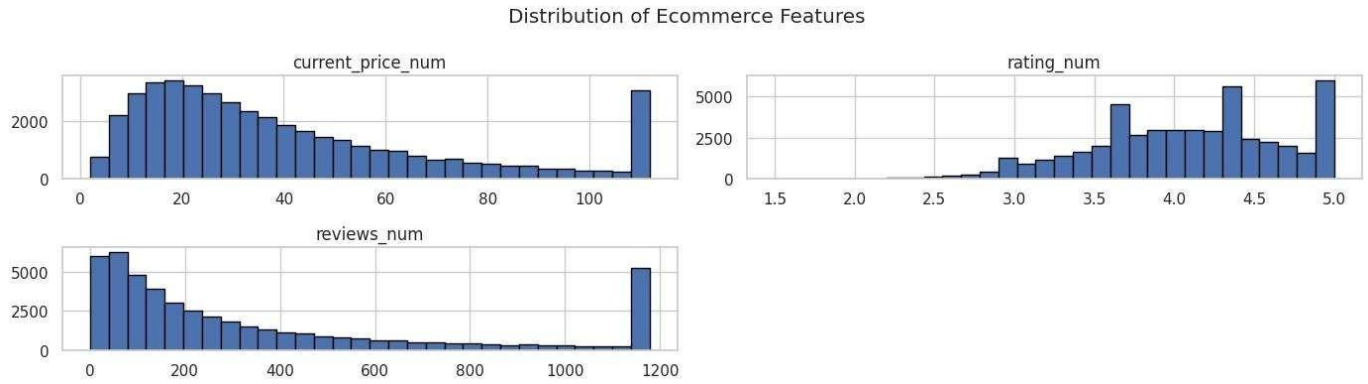
Insight: The scaled values make numerical features easier to compare on a common scale. The original columns are retained in `df`, while the transformed columns are stored in `df_scaled_features` with the suffix `_scaled`.

```
#  Plot histograms for available numerical features

hist_cols = [
    col for col in [
        'current_price_num',
```

```
        'rating_num',
        'reviews_num'
    ]
    if col in df.columns
]

if hist_cols:
    df[hist_cols].hist(
        figsize=(14, 4),
        bins=30,
        edgecolor='black'
    )
    plt.suptitle('Distribution of Ecommerce Features')
    plt.tight_layout()
    plt.show()
else:
    print("No suitable columns available for histograms.")
```



Histogram — Theory

A histogram displays the frequency distribution of a numerical feature by grouping values into intervals. It helps identify common value ranges, skewness, and the spread of the data.

Insight: The histograms show how product prices, ratings, and review counts are distributed. A long tail indicates that a small number of products have values much higher or lower than most other products.

```
# Box plots for two or three numerical features

box_cols = [
    col for col in [
        'current_price_num',
        'reviews_num',
        'rating_num'
    ]
    if col in df.columns
]

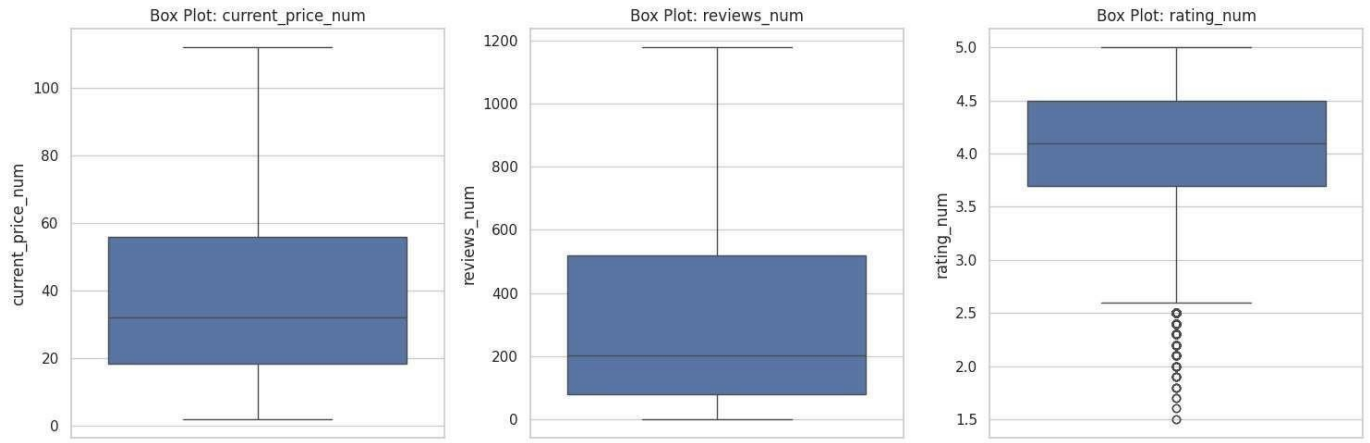
if box_cols:
    fig, axes = plt.subplots(
        1, len(box_cols),
        figsize=(5 * len(box_cols), 5)
    )

    if len(box_cols) == 1:
```

```
        axes = [axes]

    for ax, col in zip(axes, box_cols):
        sns.boxplot(y=df[col], ax=ax)
        ax.set_title(f'Box Plot: {col}')

    plt.tight_layout()
    plt.show()
else:
    print("No suitable columns available for box plots.")
```



Box Plot — Theory

A box plot displays the median, quartiles, spread, and potential outliers of a numerical feature. The box represents the interquartile range, while the line inside the box represents the median.

Insight: Box plots help compare the spread of product prices and review counts and show whether the data contains extreme values. The box plots in the outlier-treatment cell show the effect of IQR capping; this cell provides a separate visualization of selected features.

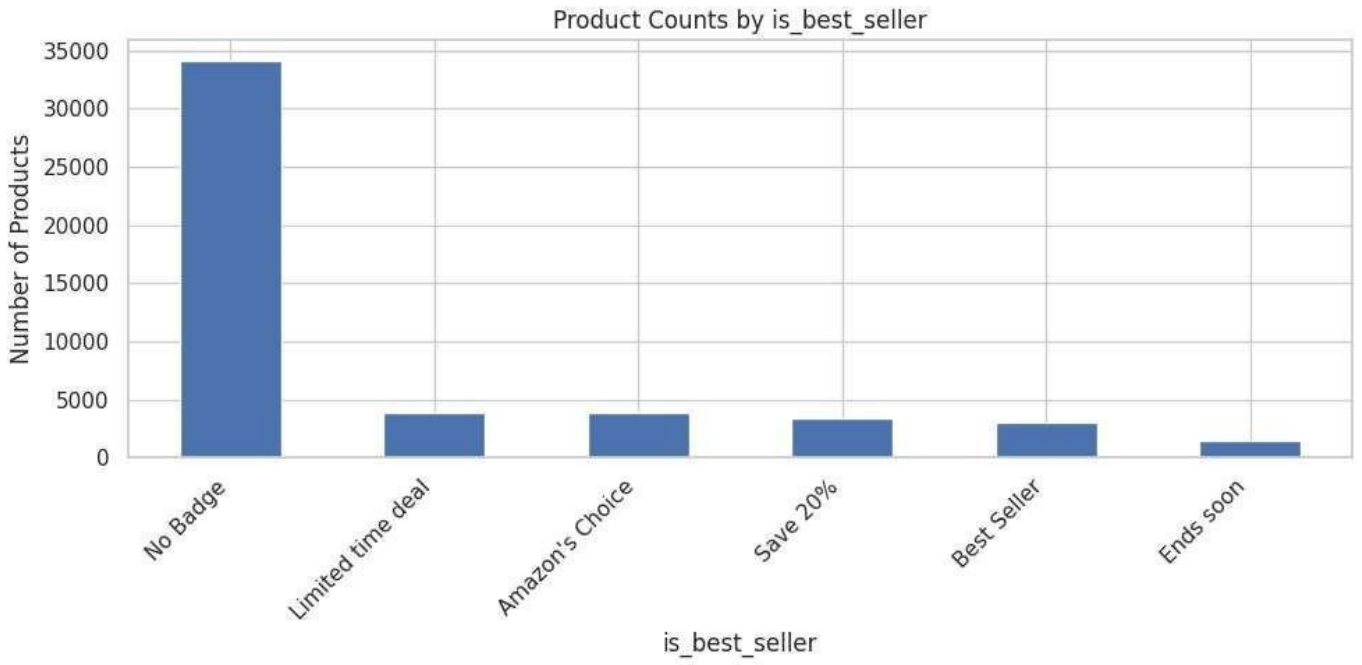
```
# Count products by category or best-seller status

bar_col = find_column([
    'best_seller_category',
    'is_best_seller',
    'category',
    'product_category'
])

if bar_col:
    counts = df[bar_col].fillna('Unknown').astype(str).value_counts().head(15)

    plt.figure(figsize=(10, 5))
    counts.plot(kind='bar')
    plt.title(f'Product Counts by {bar_col}')
    plt.xlabel(bar_col)
    plt.ylabel('Number of Products')
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()
```

```
else:
    print("No category or best-seller column found for the bar chart.")
```



Bar Chart — Theory

A bar chart compares values or counts across categories. It is useful for understanding how products are distributed among groups, such as product categories or best-seller status.

Insight: The bar chart shows which categories contain more or fewer products. Differences in counts can help identify the most common groups in the dataset.

```
#  Scatter plots of price against other features

scatter_pairs = [
    ('current_price_num', 'rating_num'),
    ('current_price_num', 'reviews_num'),
    ('current_price_num', 'bought_last_month_num')
]

valid_pairs = [
    pair for pair in scatter_pairs
    if pair[0] in df.columns and pair[1] in df.columns
]

if valid_pairs:
    plot_df = df.sample(
        n=min(3000, len(df)),
        random_state=42
    )

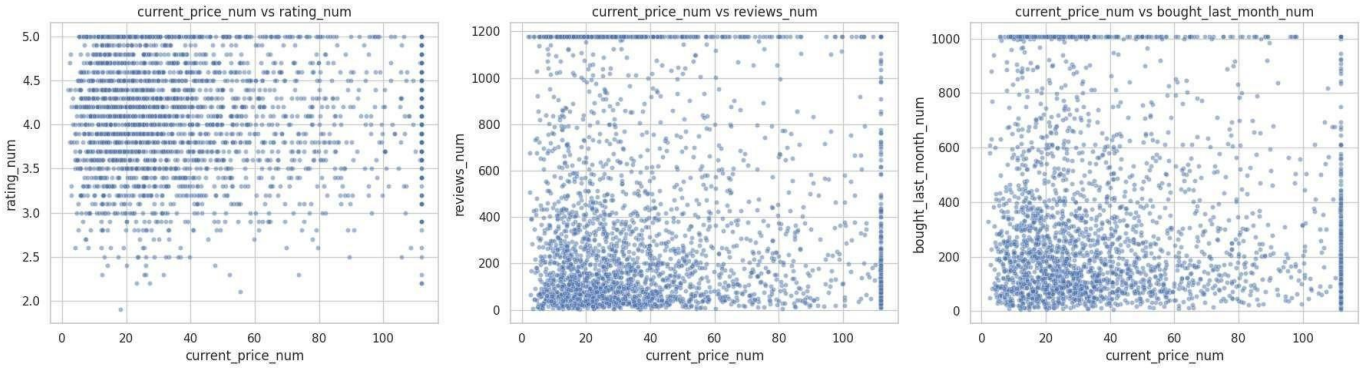
    fig, axes = plt.subplots(
        1, len(valid_pairs),
        figsize=(6 * len(valid_pairs), 5)
    )

    if len(valid_pairs) == 1:
        axes = [axes]

    for ax, (x_col, y_col) in zip(axes, valid_pairs):
```

```
        sns.scatterplot(
            data=plot_df,
            x=x_col,
            y=y_col,
            alpha=0.5,
            s=20,
            ax=ax
        )
        ax.set_title(f'{x_col} vs {y_col}')

    plt.tight_layout()
    plt.show()
else:
    print("No valid feature pairs found for scatter plots.")
```



Scatter Plot — Theory

A scatter plot displays the relationship between two numerical features. Each point represents an observation, allowing us to examine trends, clusters, and possible relationships.

Insight: Comparing price with rating, reviews, or purchase counts can help reveal whether the features appear related. A visible pattern suggests a possible relationship, but a scatter plot alone does not prove causation.

```
# Plot product collection trends if date data exists

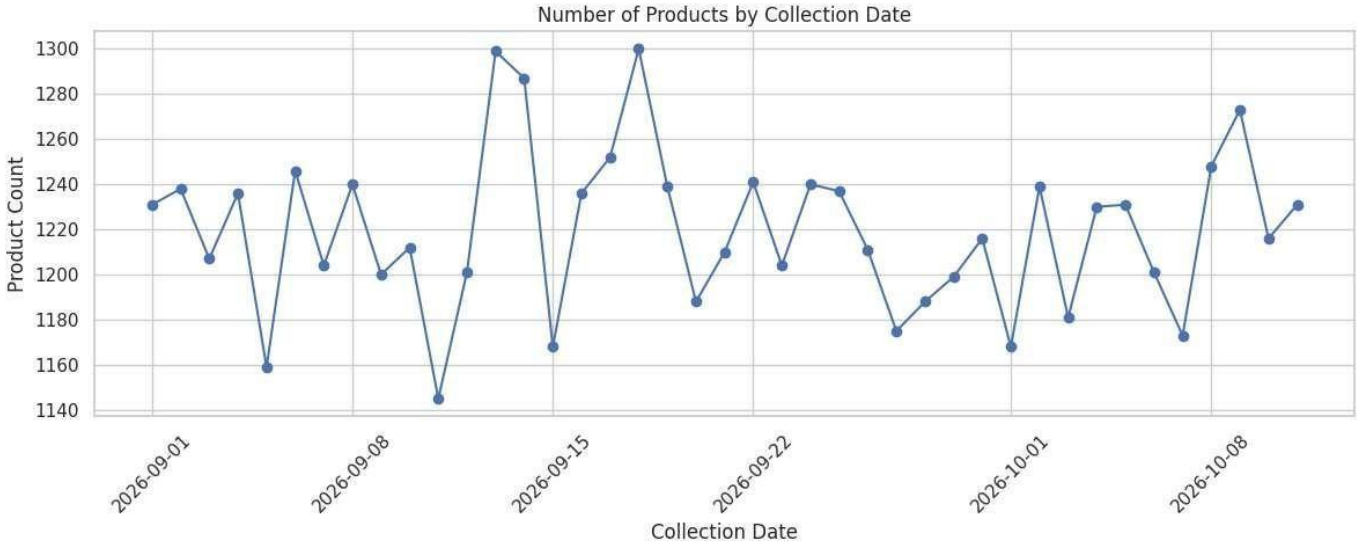
date_col = find_column([
    'collected_at',
    'collection_date',
    'date',
    'timestamp'
])

if date_col:
    date_values = pd.to_datetime(
        df[date_col],
        errors='coerce',
        utc=True
    )

    trend_df = pd.DataFrame({'date': date_values})
    trend_df = trend_df.dropna()

    if not trend_df.empty and trend_df['date'].nunique() > 1:
        trend_df['date'] = trend_df['date'].dt.date
        daily_counts = trend_df.groupby('date').size()
```

```
        plt.figure(figsize=(12, 5))
        daily_counts.plot(kind='line', marker='o')
        plt.title('Number of Products by Collection Date')
        plt.xlabel('Collection Date')
        plt.ylabel('Product Count')
        plt.xticks(rotation=45)
        plt.tight_layout()
        plt.show()
    else:
        print("Not enough valid dates to plot a trend.")
else:
    print("No collection-date column found. Line chart skipped.")
```



Line Chart — Theory

A line chart displays how a value changes across an ordered dimension, commonly time. It is useful for identifying increases, decreases, and fluctuations over a period.

Insight: If the dataset contains a collection-date column, the line chart can show how the number of recorded products varies by date. If no usable date column exists, the trend cannot be meaningfully plotted.

```
#  Pair plot using up to five available features

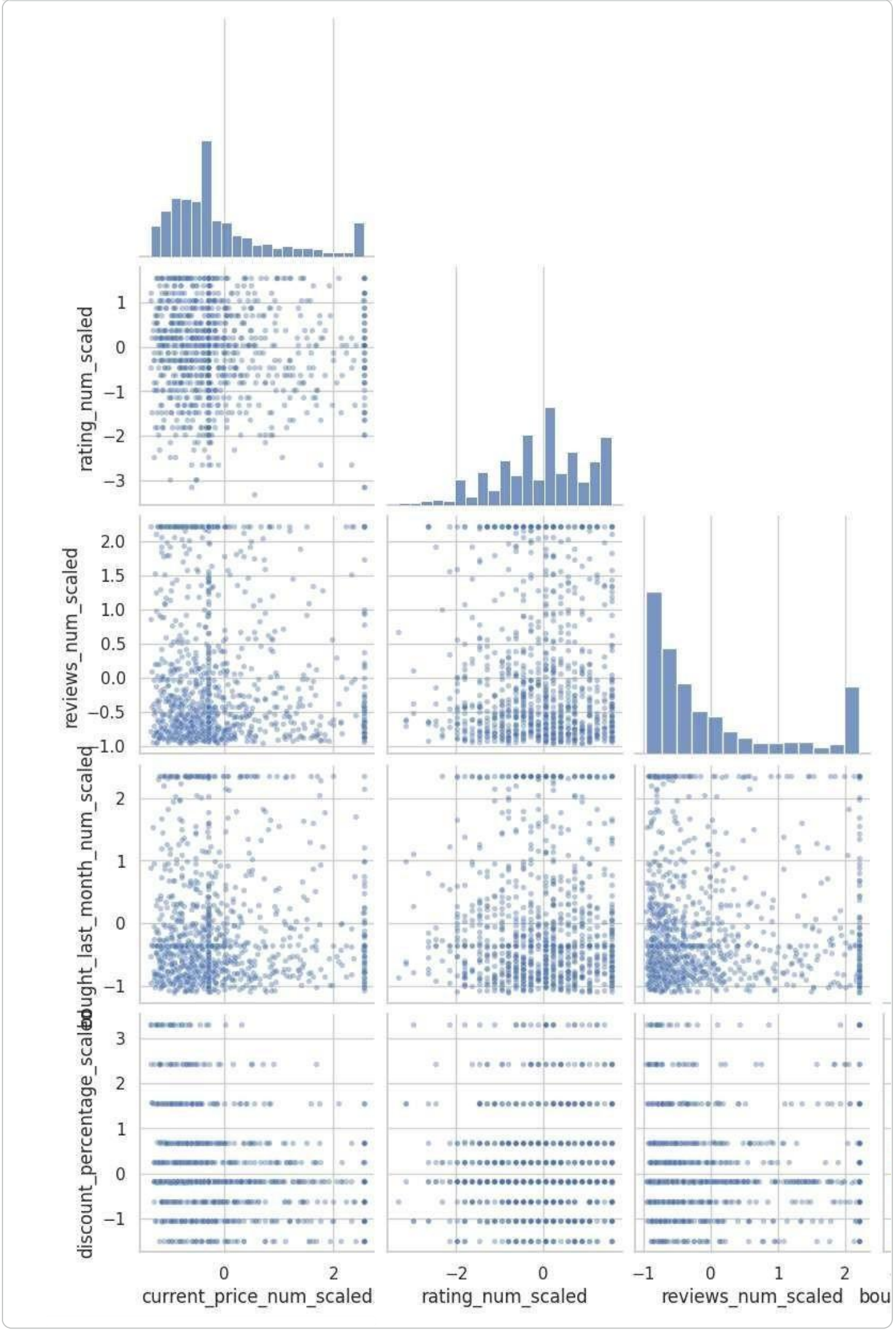
pair_candidates = [
    'current_price_num_scaled',
    'rating_num_scaled',
    'reviews_num_scaled',
    'bought_last_month_num_scaled',
    'discount_percentage_scaled'
]

pair_cols = [
    col for col in pair_candidates
    if col in df_scaled_features.columns
]

# Need at least two available features
if len(pair_cols) >= 2:
```

```
    pairplot_data = df_scaled_features[pair_cols].copy()
    pairplot_data = pairplot_data.sample(
        n=min(1000, len(pairplot_data)),
        random_state=42
    )

    sns.pairplot(
        pairplot_data,
        diag_kind='hist',
        corner=True,
        plot_kws={'alpha': 0.4, 's': 15}
    )
    plt.show()
else:
    print("At least two scaled features are required for a pair plot.")
```


rating_num_scaled
reviews_num_scaled
bought_last_month_num_scaled
discount_percentage_scaled
current_price_num_scaled
rating_num_scaled
reviews_num_scaled
bou

Pair Plot — Theory

A pair plot displays pairwise relationships among multiple numerical features. The diagonal usually shows the distribution of each feature, while the other panels show relationships between pairs.

Insight: The pair plot helps examine several features together and identify possible patterns, clusters, and correlations. A sample is used when the dataset is large to reduce plotting time.

```
#  Correlation heatmap for scaled features

if len(df_scaled_features.columns) >= 2:
    correlation_matrix = df_scaled_features.corr()

    plt.figure(figsize=(10, 7))
    sns.heatmap(
        correlation_matrix,
        annot=True,
        cmap='coolwarm',
        fmt='.2f',
        vmin=-1,
        vmax=1
    )

    plt.title('Correlation Heatmap of Scaled Features')
    plt.tight_layout()
    plt.show()
else:
    print("At least two scaled features are required for a heatmap.")
```

